# Fine-tune SmolVLA trên LIBERO-Long bằng học tăng cường

Study · Reproduce · Analyse · Improve

Tên, lớp, email

Ngày nộp

**Tóm tắt nội dung**

Chúng tôi tái lập `lerobot/smolvla_libero` (SmolVLA 0,45B) trên LIBERO-10, phân tích thất bại, rồi fine-tune bằng RL on-policy. **Reproduce:** với `n_action_steps` = 10 (NAS), mô hình đạt **72,8%** (1092/1500, 3 seed); cùng checkpoint nhưng NAS = 1 hoặc 50 chỉ đạt 54,6% và 47,8%. **Analyse:** thất bại tập trung ở task hai vật (task 0, 8): xong bước con đầu rồi kẹt, gắp/đặt thiếu chính xác. **Improve:** sau một vòng sàng lọc (gán nhãn advantage kiểu RECAP, AWR, FPO), chúng tôi phát triển FPO (Flow Policy Optimization) qua 4 phiên bản, mỗi phiên bản đổi đúng một nhóm yếu tố và được đánh giá theo cặp (McNemar chính xác) trên 300 episode task 0/8 và 160 episode của 8 task không train. FPO v4 (trọng số advantage theo segment) tăng tổng task 0+8 từ 40,3% lên **46,7%** ($p = 0{,}025$), task 8 từ 40,0% lên 53,3% ($p = 0{,}002$), giữ task 0 (40,0% so với 40,7%) và không làm giảm có ý nghĩa 8 task còn lại (78,1% so với 79,4%, $p = 0{,}83$); cả ba dự đoán ghi trước khi train đều đúng. Mức thay đổi trọng số rất nhỏ ($< 0{,}15\%$), và trên init chưa train mức tăng chưa có ý nghĩa thống kê.

## 1 Study: RL cho VLA và chính sách flow

**Năm hướng RL cho VLA.** Bảng 1 tóm tắt các bài đã đọc (số liệu ghi theo vị trí trong bài, chi tiết ở `docs/study.md`).

| Bài | Model nền | Mục tiêu học | Kết quả chính trên LIBERO |
|---|---|---|---|
| SimpleVLA-RL | OpenVLA-OFT 7B (token) | GRPO, reward nhị phân, bỏ nhóm toàn thắng/thua | TB 91,0→99,1; SFT 1 traj.: Long 17,3→91,7 |
| $\pi_{\text{RL}}$ | $\pi_0$, $\pi_{0.5}$ (flow) | PPO; log-prob qua Flow-Noise hoặc Flow-SDE | few-shot $\pi_0$: 57,6→97,6 |
| $\pi^*_{0.6}$ / RECAP | $\pi_{0.6}$ (flow) | điều kiện policy theo chỉ báo advantage, không cần log-prob | robot thật: throughput $>2\times$ |
| RIPT-VLA | QueST, OpenVLA-OFT | PPO + RLOO, $K = 16$ | QueST: 82,7→93,6; OFT: 96,7→97,5 |
| VLA-RL | OpenVLA-7B (token) | PPO + reward model quá trình | TB 76,5→81,0 |

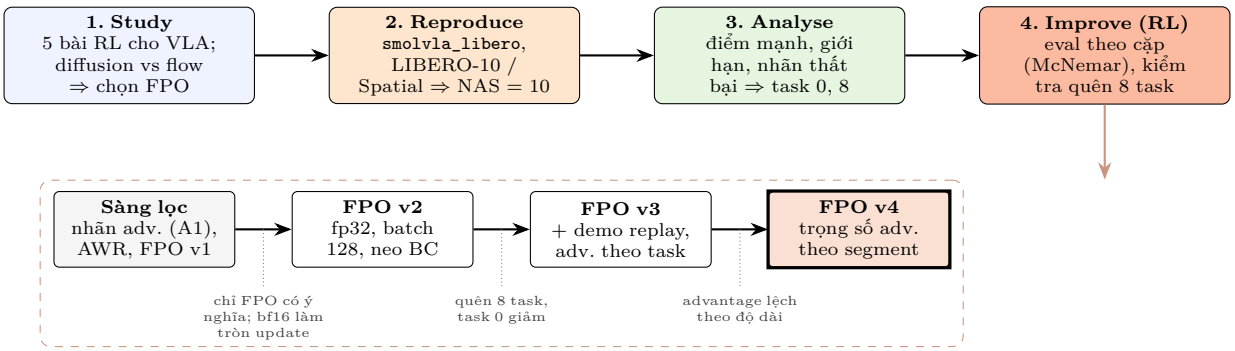
**Bảng 1:** Năm phương pháp RL cho VLA (nguồn: `docs/study.md` §1–2).

**Diffusion và flow matching.** SmolVLA dùng flow matching: $x_\tau = \tau\varepsilon + (1-\tau)a$, mạng học vận tốc $v_\theta \approx \varepsilon - a$, và sinh hành động bằng 10 bước Euler từ nhiễu. Ngoài nhiễu ban đầu, quá trình sinh là tất định, nên $\log \pi_\theta(a \mid o)$ không tính rẻ được, trong khi policy gradient cần nó. Có ba lối ra: (i) thêm nhiễu vào từng bước Euler để mỗi bước là một Gauss (Flow-Noise/Flow-SDE, $\pi_{\text{RL}}$); (ii) bỏ log-prob, điều kiện policy theo advantage (RECAP); (iii) xấp xỉ tỉ lệ likelihood bằng chênh lệch loss flow matching (FPO):

$$\hat{r}(\theta) = \exp\big(c\,[\hat{\mathcal{L}}_{\theta_{\text{old}}}(a;o) - \hat{\mathcal{L}}_\theta(a;o)]\big), \qquad J = \mathbb{E}\big[\min(\hat{r}\hat{A},\ \text{clip}(\hat{r}, 1\pm\epsilon)\hat{A})\big], \tag{1}$$

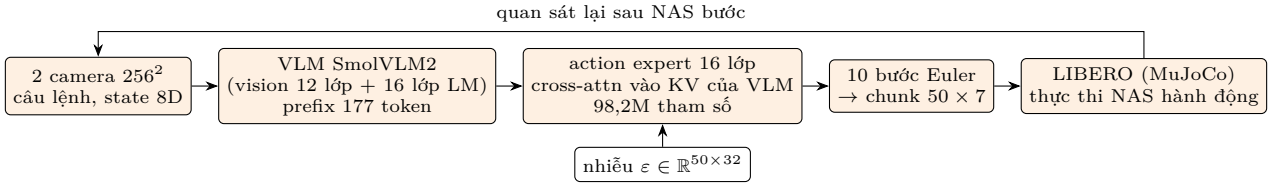
với $\hat{\mathcal{L}}$ tính trên cùng các cặp $(\tau, \varepsilon)$ đã lưu khi thu rollout. **Lựa chọn:** chúng tôi thử (ii) và (iii) vì chúng không sửa bộ sinh hành động và vừa một GPU 16 GB; (iii) là hướng chính. (i) chỉ dừng ở mức đề xuất.

**[Cần bạn điền / kiểm tra]** Kiểm tra lại các số trích từ bài báo trong Bảng 1 và định nghĩa tỉ lệ FPO gốc (có hệ số scale $c$ hay không, loss là mean hay sum theo chiều action; xem `results/round2/fpo_v3/REVIEW_CHECKLIST.md` mục 1).




**Hình 1:** Luồng công việc. Kết quả của mỗi phần quyết định cấu hình của phần sau (FPO từ Study, NAS=10 từ Reproduce, task 0 và 8 từ Analyse). Hàng dưới: chuỗi Improve; chữ nhỏ là chẩn đoán dẫn tới phiên bản kế tiếp.

# 2 Reproduce

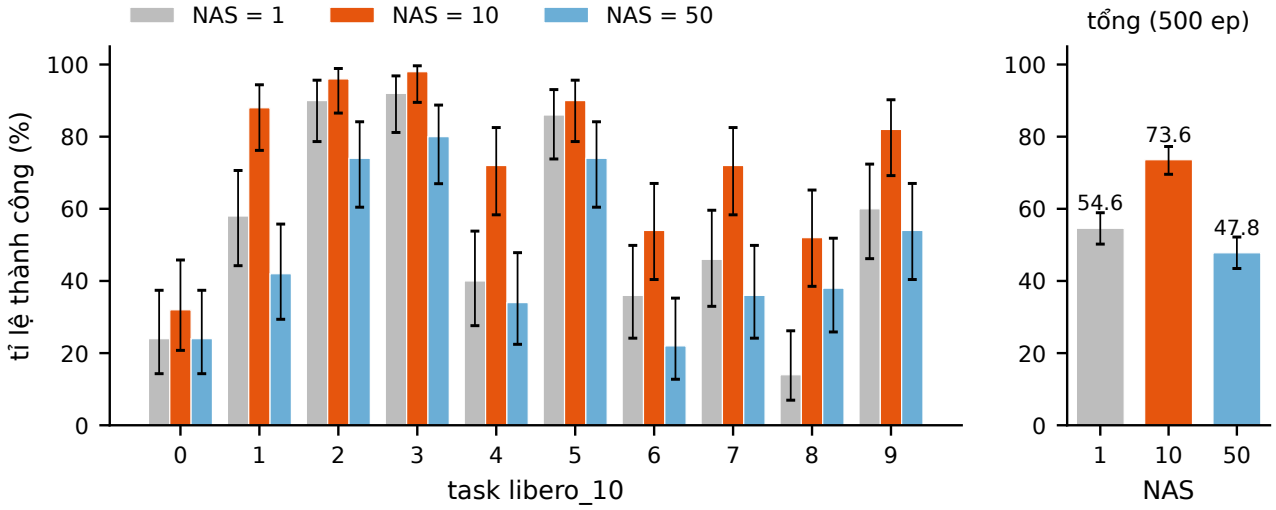



**Hình 2:** Pipeline suy luận của `lerobot/smolvla_libero` (450,0M tham số; 96,5 ms/chunk trên RTX 5070 Ti; nguồn: `results/probe_report.json`).

**Thiết lập.** Checkpoint `lerobot/smolvla_libero` (commit `a24f1f38`), LeRobot 0.6.2, 1×RTX 5070 Ti 16 GB, `control_mode=relative`, giới hạn 520 bước (LIBERO-10). Episode $k$ luôn dùng init state $k$; seed chỉ đổi nhiễu lấy mẫu. Thành công là khi mọi predicate BDDL đúng. Báo khoảng Wilson 95%.

| Run | Suite | NAS | Episode | Thành công (Wilson 95%) |
|---|---|---|---|---|
| B_main | LIBERO-10 | 1 | 3 seed × 10 init × 10 task | 158/300 = 52,7% (47,0–58,2) |
| D_nas1 | LIBERO-10 | 1 | 1 seed × 50 init | 273/500 = 54,6% (50,2–58,9) |
| **D_nas10** | LIBERO-10 | **10** | 3 seed × 50 init | **1092/1500 = 72,8%** (70,5–75,0) |
| D_nas50 (mặc định config) | LIBERO-10 | 50 | 1 seed × 50 init | 239/500 = 47,8% (43,5–52,2) |
| F_spatial | LIBERO-Spatial | 1 | 3 seed × 10 init | 237/300 = 79,0% (74,0–83,2) |
| F_spatial_{nas10, nas50, abs} | LIBERO-Spatial | 10 / 50 / 1† | 1 seed × 10 init | 82% / 58% / 2% |

**Bảng 2:** Kết quả tái lập. †`abs`: NAS=1 với `control_mode=absolute`. (Nguồn: `eval_info.json` của từng job trong `results/runs/`; `docs/reproduce.md` §4).




**Hình 3:** Cùng một checkpoint, số hành động thực thi trước khi quan sát lại (NAS) đổi kết quả tới 25 điểm. NAS=10 tốt nhất ở cả 10 task (cặp cùng init: NAS=10 thắng 118, NAS=1 thắng 23). Thanh lỗi: Wilson 95%.

**So với bài gốc.** Bài SmolVLA báo LIBERO-Long 71% và Spatial 90% đối chiếu với PDF bài gốc. Đọc theo nghĩa "dự đoán lại sau mỗi hành động" (NAS=1), chúng tôi thấp hơn (52,7% và 79,0%); với NAS=10 thì đạt 72,8% trên LIBERO-10. Giá trị 10 lấy từ tài liệu LeRobot cho Pi0.5, không phải khuyến nghị cho SmolVLA, nên từ đây mọi phần dùng NAS=10. Hai lần chạy cùng cấu hình trùng 99/100 episode. Khác biệt chính: không train lại (dùng checkpoint công bố); 2 suite thay vì 4; phải thêm `--rename_map` cho camera; `smolvla_base` không chạy được trên LIBERO (action 6 chiều). Chi tiết: `differences.md`.

# 3 Analyse

| **Điểm mạnh** | **Bằng chứng** (D10 = D_nas10, 150 ep/task) |
|---|---|
| S1 Task một bước / vật khớp nối làm tốt | task 3: 146/150, task 2: 138, task 1: 136, task 5: 135 |
| S2 Hiểu đúng câu lệnh | nhầm vật/đích (F5) chỉ 1/28 thất bại có video ở D10, 1/142 ở B_main |
| S3 Chunk + quan sát lại sửa được sai | NAS=10 cứu 10/16 lần gắp hỏng của NAS=50; 14/170 episode thành công có trượt vật rồi gắp lại |
| S4 Nhỏ, nhanh | 450M tham số, 96,5 ms/chunk; train, eval, RL đều chạy trên 1 GPU 16 GB |
| **Giới hạn** | |
| L1 Task dài: xong bước con đầu rồi dừng | hai task hai vật yếu nhất: task 0 60/150, task 8 71/150; task 8: 19/27 thất bại xong đúng một ấm |
| L2 Thao tác tinh kém | task 0: 25/29 thất bại là gắp hỏng (A); trượt vật ở 20/29 thất bại so với 5/21 thành công |
| L3 Nhạy với cách thực thi | NAS 1/10/50: 54,6/73,6/47,8%; `absolute` thay `relative`: 2/100 |
| L4 Kết quả do cảnh quyết định | 500 cảnh × 3 seed: 289 thắng cả 3, 69 thua cả 3, chỉ 142 lẫn lộn |
| L5 Khó fine-tune bằng recipe mặc định | expert nạp bf16: lr $10^{-5}$, 6000 bước chỉ đổi 5,5% phần tử expert (Mục 4.2) |

**Bảng 3:** Điểm mạnh và giới hạn (nguồn: `docs/analyse.md`, `docs/failure_overview.md`; nhãn bằng mắt F0–F6 do một người gán, nhãn tự động A–G theo ngưỡng, xem `improve/src/failure_labels.py`).




**(a)** Task 0: lon xanh đã vào giỏ, sau đó cầm lon đỏ lơ lửng tới hết giờ (F3, L1).




**(b)** Task 8: ấm thứ hai bị kẹp nằm ngang (F2, L2).

**Hình 4:** Hai kiểu thất bại điển hình (B_main, seed 1000; 4 khung: bước 10, 1/3 episode, lúc dừng, cuối).

**Hệ quả cho Improve.** Lỗi do cách thực thi đã được sửa bằng NAS=10; L1 và L2 thì không. Vì vậy Improve tập trung vào **task 0** (hai lon vào giỏ) và **task 8** (hai ấm lên bếp), hai task có tỉ lệ thành công 30–70%, đủ đa dạng để advantage theo nhóm có tín hiệu. Reward được thiết kế từ phân tích này: +1 khi thành công, shaping theo tiến độ $\Phi$ (số predicate con đã đúng), trừ điểm khi làm rơi, trượt, va chạm và hết giờ (`improve/src/reward_r2.py`).

# 4 Improve: RL on-policy cho SmolVLA

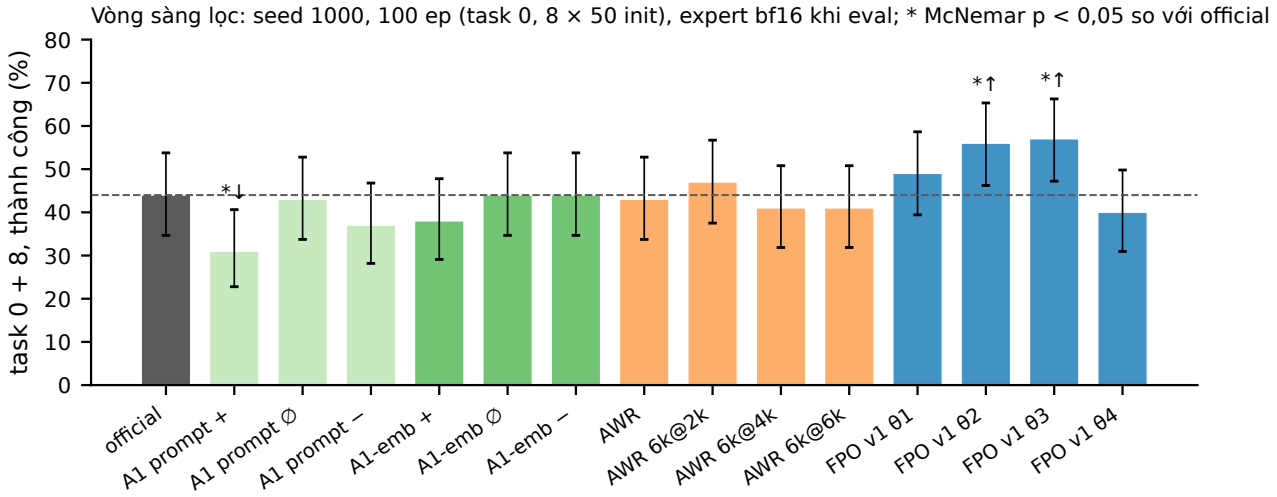
## 4.1 Giao thức chung

- **Thu rollout:** mỗi vòng 2 task × 16 init train × 8 rollout = 256 episode bằng policy hiện tại. Advantage tính theo *segment* giữa các mốc tiến độ: $A = G_{\text{seg}} + \gamma^L V_{k+1} - V_k$, với $V_k$ là trung bình return của các rollout cùng nhóm tại mốc $k$; mỗi quyết định (chunk) nhận advantage của segment chứa nó.
- **Đánh giá theo cặp:** task 0, 8 × 50 init × seed 3000/4000/5000 (300 cặp/arm; 34 init/task chưa dùng để train = *held-out*); quên: 8 task còn lại × 20 init, seed 1000. Cặp = cùng (seed, task, init); McNemar chính xác hai phía; không hiệu chỉnh nhiều phép so sánh.
- **Chọn checkpoint ghi trước:** giữ $\theta_4$ trừ khi tỉ lệ thành công lúc thu rollout giảm $> 15$ điểm giữa hai vòng; không chọn theo eval.

## 4.2 Vòng sàng lọc

Ba họ phương pháp được so trên cùng giao thức (seed 1000, 100 episode, expert bf16 khi eval): **A1** gắn nhãn advantage positive/negative vào prompt hoặc qua embedding học được (kiểu RECAP), **A2** AWR (loss flow có

trọng số advantage), và **FPO v1** (phương trình 1). Chỉ FPO v1 $\theta_2$, $\theta_3$ tốt hơn official có ý nghĩa (56% và 57% so với 44%, $p = 0{,}01$); A1 prompt positive *kém hơn* ($p = 0{,}04$); nhãn positive và negative không tách nhau (chênh $-6$ điểm, $p = 0{,}29$). Nguyên nhân khả dĩ cho AWR/A1: action expert được nạp ở bf16, bước cập nhật nhỏ hơn độ phân giải bf16 (sau 6000 bước chỉ 5,5% phần tử của expert thay đổi). Hai phát hiện này dẫn tới FPO v2.




**Hình 5:** Vòng sàng lọc (nguồn: `results/round2/episodes.csv`). Mũi tên ↑ / ↓: tốt hơn/kém hơn official có ý nghĩa ($p < 0{,}05$).

## 4.3 Chuỗi phiên bản FPO

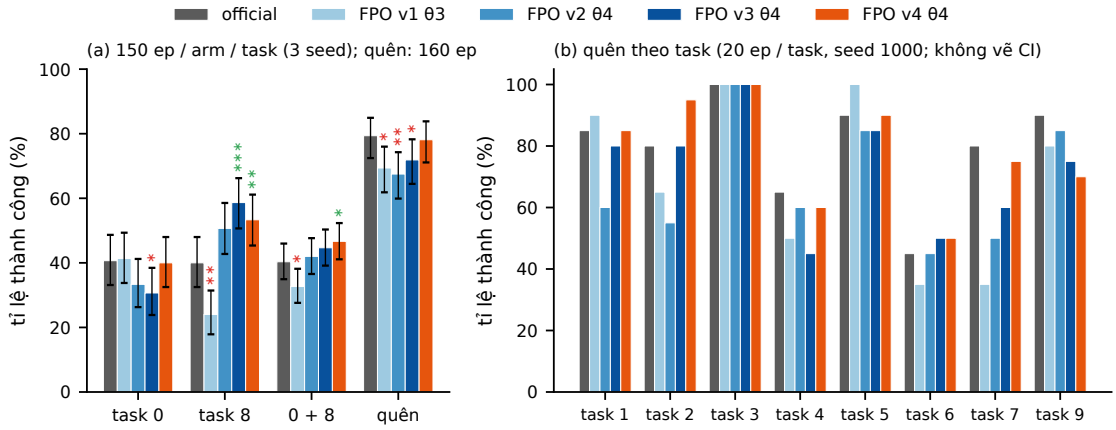
| Bản | Thay đổi so với bản trước | Lý do (chẩn đoán) | Bước grad. |
|---|---|---|---|
| v1 | FPO, batch 32, expert bf16 | — | 262 |
| v2 | trọng số chính fp32, eval expert fp32; batch 128; neo BC $\lambda = 5$ trên rollout thành công; dừng sớm khi clip_frac $> 0{,}2$; tối đa 3 epoch | cập nhật bị làm tròn ở bf16; tỉ lệ $\hat{r}$ nhiễu ở batch 32 | 84 |
| v3 | + demo replay LIBERO-10 (32 mẫu/bước, $\lambda = 5$, cả 10 task); chuẩn hoá advantage theo task; batch cân bằng 64/64 | v2 làm quên 8 task ($p = 0{,}003$) và giảm task 0 | 402 |
| v4 | trọng số $w = 1/n_{\text{decision}}$(segment), không trừ mean | ở v3, 44–53% quyết định của rollout *thất bại* ở task 0 có $\hat{A} > 0$ (Hình 8) | 228 |

**Bảng 4:** Mỗi phiên bản giữ nguyên mọi thứ khác ( 0, task, init, 4 vòng, 256 rollout/vòng, reward, seed thu rollout, quy tắc chọn). Cấu hình: `results/round2/configs/fpo\{,_v2,_v3,_v4\}.yaml`.

## 4.4 Kết quả

| | official | FPO v1 $\theta_3$ | FPO v2 $\theta_4$ | FPO v3 $\theta_4$ | **FPO v4 $\theta_4$** |
|---|---|---|---|---|---|
| task 0 (150) | 61 | 62 | 50 | 46 ▼ | **60** |
| task 8 (150) | 60 | 36 ▼ | 76 | 88 ▲ | **80** ▲ |
| tổng 0+8 (300) | 40,3% | 32,7% ▼ | 42,0% | 44,7% | **46,7%** ▲ |
| v4 vs official (chỉ v4 / chỉ off.) | | | | | 42/23, $p = 0{,}025$ |
| held-out 0+8 (204) | 84 | 66 ▼ | 86 | 87 | 91 |
| quên, 8 task (160) | 127 | 111 ▼ | 108 ▼ | 115 ▼ | **125** |
| v4 vs official | | | | | 10/12, $p = 0{,}83$ |

**Bảng 5:** Giao thức cuối (expert fp32). ▲/▼: khác official có ý nghĩa ($p < 0{,}05$, McNemar theo cặp). v4 so với v3: task 0 21/7 ($p = 0{,}013$), task 8 17/25 ($p = 0{,}28$), quên 17/7 ($p = 0{,}064$). Nguồn: `results/round2/fpo_v4/pooled.md`; v1 tính từ `improve/outputs/fpo_v2/eval_s*/fpo_v1_it3`.

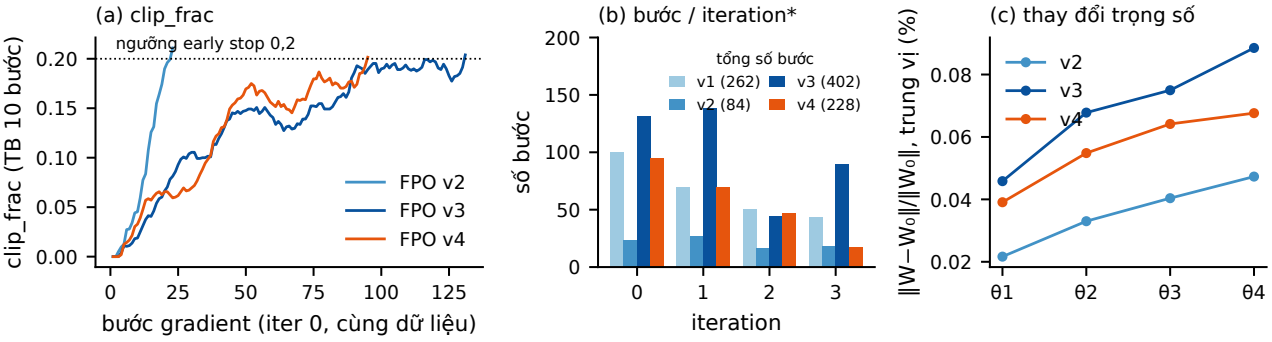



Sao: McNemar chính xác so với official (cặp cùng seed, task, init); xanh = tốt hơn, đỏ = kém hơn; * p<0,05, ** p<0,01, *** p<0,001; không hiệu chỉnh nhiều phép so sánh.

**Hình 6:** So sánh cuối. (a) Thanh lỗi Wilson 95%; sao: McNemar so với official. (b) Quên theo task (20 episode/task).

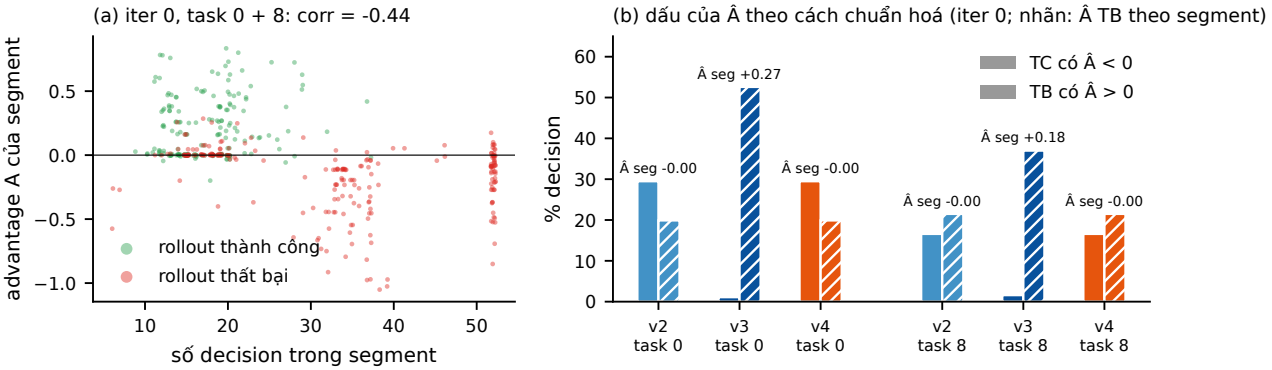
**Dự đoán ghi trước (v4).** Trước khi train v4, chúng tôi ghi (có timestamp, `docs/WORKLOG.md`) giả thuyết "task 0 giảm ở v2/v3 một phần do advantage lệch theo độ dài" và ba dự đoán với tiêu chí cố định: (a) task 0 không còn kém official ($p > 0{,}05$) và cao hơn v3: **đúng** (60 vs 61, $p = 1{,}0$; $60 > 46$); (b) task 8 vẫn hơn official có ý nghĩa: **đúng** ($p = 0{,}002$); (c) quên không tệ hơn v3: **đúng** ($125 \geq 115$). Trên init held-out, v4 chưa khác official có ý nghĩa (91 vs 84/204, $p = 0{,}32$; task 8 held-out 53 vs 45/102, $p = 0{,}13$).

## 4.5 Chẩn đoán




* v1: batch 32; v2–v4: batch 128 (v3, v4: 64 / task).

**Hình 7:** (a) Trên cùng dữ liệu iter 0, clip_frac của v2 vượt ngưỡng sau 23 bước; v3/v4 tăng chậm hơn nên chạy lâu hơn. (b) Mọi vòng đều dừng sớm. (c) Trọng số chỉ thay đổi 0,05–0,09% (trung vị theo tensor) so với $\theta_0$.



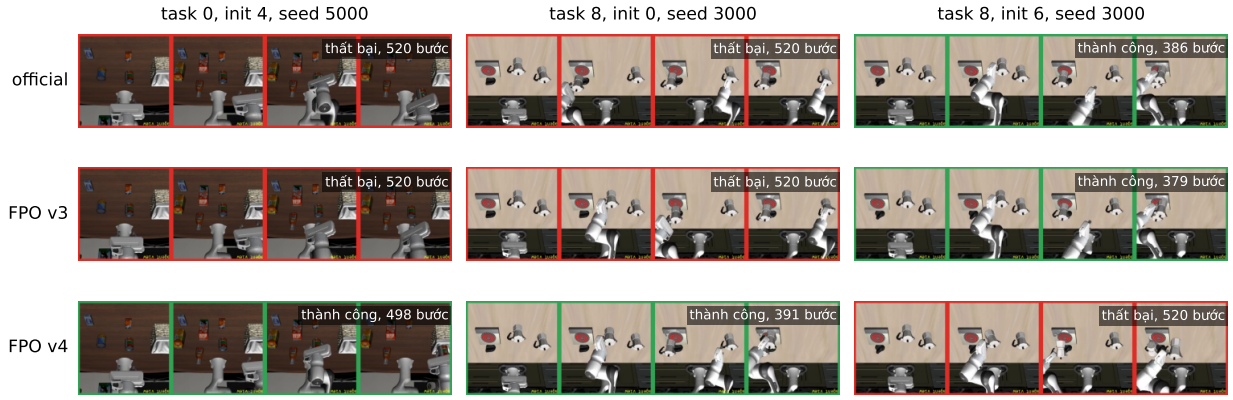

**Hình 8:** (a) Rollout thất bại luôn chạy hết 520 bước nên segment cuối dài và có advantage âm (tương quan −0,44): trung bình theo quyết định bị kéo âm dù trung bình theo segment ≈ 0. (b) v3 trừ mean theo quyết định nên 52,6% quyết định của rollout thất bại ở task 0 có $\hat{A} > 0$; v4 giữ dấu advantage gốc và trung bình theo segment ≈ 0.

- **Update rất nhỏ.** Tỉ lệ FPO được scale bởi $c = 1/\overline{\mathcal{L}}_{\text{old}} \approx 24{,}6$; một mẫu bị clip khi loss đổi $> 0{,}0074$ (18% loss trung bình), nên mọi vòng dừng sớm sau 16–138 bước với lr $10^{-6}$. Trọng số chỉ dịch dưới 0,15%

(Hình 7c). Nạp checkpoint cho eval đã được kiểm tra: 155/155 tensor đã train khớp từng bit ở fp32.

- **Advantage thô đúng** (mọi quyết định khớp đúng rollout và segment; trung bình theo segment $|\bar{A}| \leq 0{,}0023$); độ lệch nằm ở *trọng số theo quyết định* (Hình 8).
- **Kiểu thất bại** (nhãn tự động, chưa xem video): ở task 8, số thất bại "chưa ấm nào lên bếp" là 14 (official), 50 (v2), 13 (v3), 12 (v4); thất bại của v4 chủ yếu là "xong một ấm" (55/70).

### 4.6 Ví dụ định tính




**Hình 9:** Cùng (seed, task, init), replay chính xác từ eval gốc (kiểm tra `replay_check_*.csv`): official, v3, v4. Mỗi dải: 4 khung đều nhau của camera agent (xoay 180° như policy nhìn thấy); viền xanh = thành công, đỏ = thất bại. Video: `results/videos/v4_paired/`.

**[Cần bạn điền / kiểm tra]** Mô tả ngắn hành vi khác nhau giữa v4 và official/v3 sau khi xem video `results/videos/v4_paired/*.mp4`.

## 5 Hạn chế và kết luận

- Mỗi phiên bản chỉ train một lần (một seed train); khác biệt giữa các phiên bản có thể một phần do ngẫu nhiên khi train.
- p-value theo tập con không hiệu chỉnh; mức tăng trên init held-out chưa có ý nghĩa.
- Cơ chế khiến v3 chạy nhiều bước hơn v2 (trừ mean hay số hạng demo) chưa được tách bằng ablation.
- Nhãn thất bại A–G là tự động; nhãn F0–F6 do một người gán từ khung ảnh.
- Chỉ 2/10 task được train; chưa thử Flow-SDE (log-prob chính xác).

**Hướng tiếp theo: DSRL** (Wagenmaker và cộng sự, 2025). Giữ nguyên SmolVLA và học một policy nhỏ chọn nhiễu ban đầu $\varepsilon$ của flow matching. Hướng này không cần gradient qua VLA. Báo cáo này chưa đánh giá DSRL.

**[Cần bạn điền / kiểm tra]** Kết luận và hướng tiếp theo (ví dụ: scale tỉ lệ FPO, lặp seed train, Flow-SDE).

## Tài liệu tham khảo


- Shukor và cộng sự (2025). SmolVLA. arXiv:2506.01844.
- McAllister và cộng sự (2025). Flow Matching Policy Gradients (FPO). arXiv:2507.21053.
- Chen và cộng sự (2025). $\pi_{\text{RL}}$: Online RL fine-tuning for flow-based VLA. arXiv:2510.25889.
- Physical Intelligence (2025). $\pi^*_{0.6}$ / RECAP. arXiv:2511.14759.
- SimpleVLA-RL (2025). arXiv:2509.09674.
- RIPT-VLA (2025). arXiv:2505.17016.
- VLA-RL (2025). arXiv:2505.18719.
- Wagenmaker và cộng sự (2025). Steering diffusion policy with latent space RL (DSRL). arXiv:2506.15799.
- Liu và cộng sự (2023). LIBERO. NeurIPS Datasets and Benchmarks.

**[Cần bạn điền / kiểm tra]** Kiểm tra tên tác giả, năm và venue của từng tài liệu.